

1. The Long-Task Memory Problem

Long agent tasks accumulate decisions, failed hypotheses, benchmark results, temporary constraints, and newly discovered architecture. If all of this must be repeatedly compressed into one summary, important details can disappear. If all raw history stays equally salient, the agent can drown in it.

GPT-6 Astra introduces a different Codex mechanism: notes can persist across context windows while earlier context remains searchable. The system can preserve accumulated details without treating one compacted summary as the only surviving representation.

2. Notes versus Searchable History

Persistent notes and searchable old context solve different problems. Notes represent facts the agent expects to need again. Searchable history is an archive that can recover a detail not promoted into notes.

This resembles engineering practice: a working notebook contains current decisions, while logs and version history retain deeper evidence. The existence of logs does not remove the need for a concise current state.

3. What Belongs in Notes

Useful long-task notes include:

- current goal and finish condition;
- architecture facts that changed the plan;
- failed approaches and the evidence against them;
- commands or environments needed to reproduce important results;
- decisions that later work must preserve;
- open questions whose answers determine the next branch.

Do not record every command. A note should reduce future search cost or prevent repeated failure.

4. What Still Belongs in the Repository

Conversational memory is not a substitute for durable project truth. If future contributors need a command, invariant, schema, or design decision, it should live in appropriate repository documentation, tests, configuration, or code. A task-specific failed experiment may remain in notes; a newly discovered required migration step should become durable project documentation if it will recur.

This division prevents a successful agent session from creating undocumented organizational knowledge.

5. Context Recovery

Suppose a refactor stalls after many hours. The agent remembers that one approach failed but not why. Searchable prior context can recover the original test output and surrounding reasoning. This is stronger than a vague note such as "approach B didn't work," because the original evidence may reveal a narrower conclusion.

The danger is confirmation bias. Search should be driven by a question: "What exact failure ruled out B?" not "Find support for our current plan."

6. Durable Human Context

Kundel describes a workflow in which Codex can learn more from memories, connected work, and computer context so the user does not have to package every detail manually. The practical gain is lower setup cost. The governance question remains: which context should the agent be allowed to use, and which actions should still require confirmation?

More context increases capability and privacy surface together. Give access because it helps a real task, not because maximum context sounds powerful.

7. A Long-Run Handoff

At a context boundary, the ideal handoff is compact but falsifiable:

Goal: current target.

State: what is true now.

Evidence: key tests or measurements.

Rejected: approaches ruled out and why.

Open: unresolved questions.

Next: one or two discriminating actions.

Pointers: files, commits, or prior-context search terms.

This preserves reasoning structure without rewriting the entire transcript.

References: OpenAI, GPT-6 Astra launch; Dominik Kundel, "How I Stopped Needing to Explain Things to Codex"; OpenAI, Codex customization.

8. Search versus Remember

Searchable prior context is strongest when the question is specific. "Find the exact benchmark result from the run before we changed the parser" can recover evidence. "Remember everything important" is not a retrieval query. The agent should promote high-value state into notes and use search for details whose future importance could not be predicted.

9. Context Poisoning by Obsolete Facts

Long history preserves mistakes as well as discoveries. An early hypothesis may remain searchable after being disproved. Notes should mark rejected claims explicitly and newer evidence should supersede older assumptions. When retrieving old context, preserve its temporal status: was this a current fact, a hypothesis, or a failure from an earlier repository state?

10. Reproducible Long Tasks

For multi-hour or multi-day work, periodically ensure that a new session could resume from the repository plus a compact state note. If continuation depends on invisible local GUI state or an unrecorded manual action, the task is brittle. Recovery is itself a property worth verifying.

Independent Open Course

<https://github.com/dicnunz/codex-astra-course>

CODEX.00 | Codex with GPT-6 Astra

Handout 14: Context across Long Tasks

Fall 2026

License and source

Structural and pedagogical adaptation of MIT OpenCourseWare 24.00 Problems of Philosophy, Fall 2019, Prof. Miriam Schoenfield.

Original course and this adaptation are licensed CC BY-NC-SA 4.0.

This course is independently produced and is not affiliated with or endorsed by MIT or OpenAI.

Linked external readings and referenced product documentation retain their own copyrights and terms.

<https://creativecommons.org/licenses/by-nc-sa/4.0/>

<https://ocw.mit.edu/courses/24-00-problems-of-philosophy-fall-2019/>